

Blocs synchronisés en Java

Fabrice.Kordon@lip6.fr



En guise d'introduction...

2

Créer des sections critiques par synchronisation

-  Retarder des séquences de code
-  Laisser d'autres séquences de code progresser
-  Exclusion mutuelle?

En Java, il est aussi possible de synchroniser des blocs

-  La portée est réduite au bloc
-  Contrôle de la granularité de la synchronisation

► Point important discuté plus loin

Syntaxe similaire

```
synchronized (unObjet) {  
    mesInstructions;  
}
```

-  unObjet est l'objet sur lequel on se synchronise

► Le verrou de unObjet sera utilisé

► Attention à ce que la référence soit stable (une constante / final)

Variante du compteur partagé protégé (classe CompteurProtege2)

3

```
class CompteurProtege2 {  
    private String monNom;  
    private int compteur = 0;  
  
    CompteurProtege2 (String n) {  
        monNom = n;  
    }  
  
    public void inc() {  
        synchronized (this) {  
            compteur++;  
            // dedans pour synchronizer les traces  
            System.out.println ("après inc, " + this.toString());  
        }  
    }  
  
    public void dec() {  
        synchronized (this) {  
            compteur--;  
            // dedans pour synchronizer les traces  
            System.out.println ("après dec, " + this.toString());  
        }  
    }  
  
    public String toString () {  
        return monNom + "=" + compteur;  
    }  
}
```


Variante du compteur partagé protégé (classe Client)

4

```
class Client implements Runnable {
    private int monNumero;
    private CompteurProtege2 monCpt;
    Client (int i, CompteurProtege2 c) {
        monNumero = i;
        monCpt = c;
    }
    public void run() {
        System.out.println (monNumero + " est lancé");
        for (int i = 1 ; i <= 4 ; i++) {
            if (monNumero % 2 == 0) {
                monCpt.inc();
            } else {
                monCpt.dec();
            }
            try { // On met la dose pour forcer une commutation;-)
                Thread.sleep (100); // millisecondes
            }
            catch (InterruptedException ie) {
                System.err.println("Sleep impossible pour " + monNumero);
            }
        }
    }
    public String toString () {
        return "client " + monNumero;
    }
}
```


Variante du compteur partagé protégé (classe «principale»)

5

```
public class CompteurPartage3 {  
    public static void main(String []args) {  
        CompteurProtege2 cpt = new CompteurProtege2("cpt");  
  
        Thread lesClients [] = new Thread [6];  
  
        for (int i = 0; i < lesClients.length ; i++) {  
            Client c = new Client (i+1, cpt);  
            lesClients [i] = new Thread(c);  
            lesClients[i].start();  
        }  
    }  
}
```


5

```
$ java CompteurPartage3
```

Entrelacement différent

```
2 est lancé
6 est lancé
1 est lancé
5 est lancé
3 est lancé
4 est lancé
après inc, cpt=1
après inc, cpt=2
    dec, cpt=1
    dec, cpt=0
    dec, cpt=-1
    inc, cpt=0
    dec, cpt=-1
    inc, cpt=0
après dec, cpt=-1
après inc, cpt=0
après inc, cpt=1
après dec, cpt=0
après inc, cpt=1
après dec, cpt=0
après dec, cpt=-1
après inc, cpt=0
après dec, cpt=-1
après inc, cpt=0
après dec, cpt=-1
après inc, cpt=0
après dec, cpt=-1
après inc, cpt=0
après inc, cpt=1
après dec, cpt=0
```

2 est lancé
après inc, cpt=1
3 est lancé
1 est lancé
4 est lancé
5 est lancé
après dec, cpt=0
cpt=1

identique

après dec, cpt=-1
après inc, cpt=0
après inc, cpt=1
après dec, cpt=0
après inc, cpt=1
après dec, cpt=0
après inc, cpt=1
après dec, cpt=0
après dec, cpt=-1
après inc, cpt=0
après inc, cpt=1
après inc, cpt=2
après dec, cpt=1
après inc, cpt=2
après dec, cpt=1
après dec, cpt=0

Résultat identique

En guise de conclusion...

6

Ressources critique et sections critiques

-  Une ressource critique est manipulée dans une section critique
-  Il faut minimiser la portée des sections critiques...

Pourquoi?

Choisir la bonne granularité dans le parallélisme

-  Maximiser le parallélisme
-  Minimiser les synchronisations

